callback **should** be called. If you want to specify the conditions under which the callback **should not** be called, then you may use the `:unless` option.

8.1 Using `:if` and `:unless` with a `Symbol`

You can associate the `:if` and `:unless` options with a symbol corresponding to the name of a predicate method that will get called right before the callback. When using the `:if` option, the callback won't be executed if the predicate method returns false; when using the `:unless` option, the callback won't be executed if the predicate method returns true. This is the most common option. Using this form of registration it is also possible to register several different predicates that should be called to check if the callback should be executed.

```
class Order < ActiveRecord::Base
  before_save :normalize_card_number, if: :paid_with_card?
end
```

8.2 Using `:if` and `:unless` with a String

You can also use a string that will be evaluated using `eval` and hence needs to contain valid Ruby code. You should use this option only when the string represents a really short condition:

```
class Order < ActiveRecord::Base
  before_save :normalize_card_number, if: "paid_with_card?"
end
```

8.3 Using `:if` and `:unless` with a `Proc`

Finally, it is possible to associate `:if` and `:unless` with a `Proc` object. This option is best suited when writing short validation methods, usually one-liners:

```
class Order < ActiveRecord::Base
  before_save :normalize_card_number,
    if: Proc.new { |order| order.paid_with_card? }
end
```

8.4 Multiple Conditions for Callbacks

When writing conditional callbacks, it is possible to mix both `:if` and `:unless` in the same callback declaration:

```
class Comment < ActiveRecord::Base
  after_create :send_email_to_author, if: :author_wants_emails?,
    unless: Proc.new { |comment| comment.article.ignore_comments? }
end
```

9 Callback Classes

Sometimes the callback methods that you'll write will be useful enough to be reused by other models. Active Record makes it possible to create classes that encapsulate the callback methods, so it becomes very easy to reuse them.

Here's an example where we create a class with an `after_destroy` callback for a `PictureFile`model:

```
class PictureFileCallbacks
  def after_destroy(picture_file)
    if File.exist?(picture_file.filepath)
      File.delete(picture_file.filepath)
    end
  end
end
```

When declared inside a class, as above, the callback methods will receive the model object as a parameter. We can now use the callback class in the model: